# Week 1: Python Basics

XU Shirong

# What is Python?

- High-level, interpreted programming language
- Easy-to-read syntax
- Widely used in data science, web development, automation, AI
- Supports multiple paradigms: procedural, object-oriented, functional

# Comments

Single-line comment:

```
# This is a comment
```

Multi-line comment:

```
"""
This is a
multi-line comment
"""
```

# Variables

No need to declare type. Examples:

```
x = 5
name = "Alice"
pi = 3.1415
```

Variable names: letters, digits, underscore; cannot start with digit.

```
x = 5
y = x
x = 6
```

What is $y$?

# Basic Data Types

- Integers: int
- Floating-point numbers: float
- Strings: str
- Boolean: bool (True/False)
- Type checking: type(variable)

# Type Conversion

Common type conversion functions: int(), float(), str(), bool()
Example:

```
x = "10"
y = int(x) + 5  # y = 15
```

Useful when reading input or combining types.

# Arithmetic Operators

Addition: +, Subtraction: -, Multiplication: *, Division: /
Integer division: //, Modulus:
Example:

```
5 + 3      # 8
5 ** 2     # 25
7 // 3     # 2
7 % 3      # 1
```

# Strings

Strings are enclosed in single or double quotes.
Concatenation: $+$, Repetition: *
Indexing: str[0], Slicing: str[1:4]

# String Methods

Common methods:

```
.upper(), .lower(), .strip(), .replace(), .split()
```

Example:

```
s = "  hello "
s.strip().upper() # "HELLO" Remove frst and last whitespace (if any)
s.replace("e","l") # replace "e" by "l"
s.split("l") # split sting by "l"
```

Practice: Count the number of "hello"

```
s = "Hello  hello HEllO,kdjsk;123123"
# How to do next
```

# Lists

Ordered, mutable collection. Examples:

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob"]
mix = ["Alice", 123]
```

Supports indexing, slicing, appending. Allow for different types of data.

```
numbers = [1, 2, 3, 4]
names = numbers
numbers.append(1)
```

What is names?

# List Methods

```
nums = [3, 1, 4]
nums.append(5)
# Add 5 to the end of the list # Result: [3, 1, 4, 5]
nums.insert(1, 10)
# Insert 10 at index 1 # Result: [3, 10, 1, 4, 5]
nums.remove(4)
# Remove the first occurrence of value 4 # Result: [3, 10, 1, 5]
nums.pop()
# Remove and return the last element # Result: [3, 10, 1]
nums.sort()
# Sort the list in ascending order # Result: [1, 3, 10]
nums.reverse()
# Reverse the list order # Result: [10, 3, 1]
```

# Tuples

Ordered, immutable collection. Example:

```
point = (2, 3)
```

Useful for fixed data and as dictionary keys. Also allow for different types of data.

```
point.count(2)
point.count(1)
```

# Dictionaries

Key-value pairs, unordered. Example:

```
person = {"name": "Alice", "age": 25}
```

Access: person["name"], modify: person["age"] = 26. We can update the dictionary as

```
person.update({"name": "Alice", "gender": 211})
```

```
person = {"name": "Alice", "age": 25}
LL = person
person.update({"name": "Alice", "gender": 211})
```

What is LL?

# Sets

Unordered collection of unique elements. Example:

```
fruits = {"apple", "banana", "orange"}
fruits_1 = {"apple", "banana", "lemon"}
print(fruits.union(fruits_1))   # union
print(fruits & fruits_1) # intersection
print(fruits.symmetric_difference(fruits_1))
```

Supports union, intersection, difference operations.

# Comparison Operators

$==, !=, >, <, >=, <=$ Example:

```
print(5 > 3)  # True
print(1==2) # False
print(1!=2) # True
```

# Logical Operators

and, or, not

Example:

```
x = True
y = False
x and y   # False
x or y    # True
not x     # False
```

# Conditional Statements

Example:

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

# For Loops

Example:

```
for i in range(5):
    print(i)
```

Iterate over lists, tuples, strings.

# While Loops

Example:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

What is the output?

# Break and Continue

Example:

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

break: exit loop, continue: skip current iteration. Does it output 3?

# Functions

Define reusable code:

```
def greet(name):
    return "Hello " + name
```

Call: greet("Alice")

# Function Arguments

Positional, keyword arguments, default values:

```
def power(x, n=2):
    return x ** n
```

Usually, we can add some description to the function

```
def power(x, n=2):
    """
    :param x:
    :param n:
    :return:
    """
    return x ** n
```

# Modules

Import existing Python code:

```
import math
math.sqrt(16)  # 4.0
```

Create custom module: save .py and import.

# Random Module

```
import random
print(random.randint(1, 10))
print(random.choice(["apple", "banana"]))
```

Sometimes we have to fix random seed to reproduce simulation results:

```
import random
random.seed(1)
print(random.randint(1, 10))
print(random.choice(["apple", "banana"]))
```

# File Input/Output

Writing to file:

```
with open("file.txt", "w") as f:
    f.write("Hello")
```

Reading from file:

```
with open("file.txt", "r") as f:
    content = f.read()
```

Many Datasets are stored in txt file. Each line is a piece of data.

```
with open("file.txt", "r") as f:
    header = f.readline().strip().split()  # ["Gender", "Age", "Name
        "]
    Content = f.readlines()
print([i.split( ) for i in Content])
```

# Read Txt

Example: Create a txt file containing

```
Name Gender Age
Mike Male 21
Rick Male 24
```

Extract Headers and Data use readline and readlines.

```
Name Gender Age
Mike Male 21
Rick Male 24
```

# List Comprehensions

Example: Create lists with for loops

```
squares = [x**2 for x in range(5)]
evens = [x for x in range(10) if x % 2 == 0]
```

# Example: Sum of Even Numbers

```
numbers = [1, 2, 3, 4, 5, 6]
even_sum = sum([x for x in numbers if x % 2 == 0])
print(even_sum)  # 12
```

# Summary

- Python basics: variables, data types, operators
- Control flow: if, for, while
- Functions, modules, file I/O
- Next: explore advanced topics like classes, numpy, pandas

# Importing Libraries

```
import numpy as np
import pandas as pd
```

Here, as xx can be avoided. They just simplify the following usage.

# Creating Arrays

```
# 1D array
a = np.array([1, 2, 3, 4])

# 2D array
b = np.array([[1, 2], [3, 4]])

# zeros and ones
c = np.zeros((2,3))
d = np.ones((3,2))

# random numbers
e = np.random.rand(2,2)
```

What is the underlying distribution of *np.random.rand*

# Array Attributes

```
arr = np.array([[1,2,3],[4,5,6]])

arr.shape      # (2, 3)
arr.size       # 6
arr.ndim       # 2
arr.dtype      # data type
```

# Indexing and Slicing

```
arr = np.array([10, 20, 30, 40, 50])

arr[0]          # 10
arr[-1]         # 50
arr[1:4]        # [20 30 40]

# 2D slicing
mat = np.array([[1,2,3],[4,5,6]])
mat[1,2]        # 6
mat[:, 1]       # [2 5]
```

# Array Operations

```
a = np.array([1,2,3])
b = np.array([4,5,6])

# element-wise
a + b          # [5 7 9]
a * b          # [4 10 18]

# broadcasting
a + 10         # [11 12 13]

# sum, mean, max, min
np.sum(a)      # 6
np.mean(b)     # 5.0
np.max(a)      # 3
np.min(b)      # 4
```

# Reshaping Arrays

```
arr = np.arange(6)    # [0 1 2 3 4 5]

arr.reshape((2,3))    # [[0 1 2]
                      #  [3 4 5]]

arr.flatten()         # [0 1 2 3 4 5]
arr.T                 # transpose (for 2D arrays)
```

# Stacking and Splitting

```
a = np.array([1,2,3])
b = np.array([4,5,6])

# horizontal stack
np.hstack([a,b])      # [1 2 3 4 5 6]

# vertical stack
np.vstack([a,b])      # [[1 2 3]
                      #  [4 5 6]]

# splitting
np.split(a, 3)        # [array([1]), array([2]), array([3])]
```

# Matrix Multiplication

Numpy offers matrix multiplication (use @)

```
a = np.array([1,2,3]) # shape (1,3)
b = np.array([[1,2,3],[1,2,3],[1,2,3]]) #shape (3,3)
print(a @  b)
```

# Creating a DataFrame

There are many ways of creating DataFrame, including (1) Dictionary (2) Array

```
data = {"Name": ["Alice", "Bob", "Charlie"],
        "Age": [25, 30, 35]}
df = pd.DataFrame(data)
print(df)
data = np.array([["Alice", "Bob", "Charlie"],[25, 30, 35]]).T
df = pd.DataFrame(data)
print(df)
```

What is the difference between these two dataframes?

# Viewing Data

```
df.head()    # first 5 rows
df.tail()    # last 5 rows
df.info()    # summary
df.describe()  # statistics
```

# Selecting Columns & Rows

```
df["Name"]
df[["Name", "Age"]]
```

```
df.iloc[0]          # first row
df.iloc[0:2]        # first two rows
df[df["Age"] > 30]  # conditional
```

Adding a new column

```
df["Salary"] = [50000, 60000, 70000]
```

Modifying Columns

```
df["Age"] = df["Age"] + 1
```

# Dropping Columns or Rows

```
df.drop("Salary", axis=1, inplace=True)
df.drop(0, axis=0)  # drop first row
```

Sorting

```
df.sort_values("Age")           # ascending
df.sort_values(["Age", "Name"]) # multiple columns
```

Filtering

```
df[df["Age"] > 30]
df[(df["Age"] > 25) & (df["Name"] != "Bob")]
```

# Handling Missing Data

```
df.isnull()
df.dropna()       # drop rows with NaN
df.fillna(0)      # replace NaN with 0
```

# Applying Functions

```
df["AgePlus5"] = df["Age"].apply(lambda x: x+5)
```

# Grouping and Aggregation in Pandas

## 1. Grouping Data

```
df.groupby("Name")["Age"].mean()
```

*Compute the mean Age for each Name.*

## 2. Multiple Aggregations

```
df.groupby("Name").agg({"Age":"mean", "Salary":"sum"})
```

*Apply different aggregation functions to multiple columns.*

## 3. Pivot Table

```
df.pivot_table(index="Name", values="Age", aggfunc="mean")
```

*Reshape and summarize data in spreadsheet-style format.*

# Combining DataFrames

## 1. Merging (SQL-style join)

```
df1 = pd.DataFrame({"ID":[1,2], "Name":["Alice","Bob"]})
df2 = pd.DataFrame({"ID":[1,2], "Salary":[50000,60000]})
pd.merge(df1, df2, on="ID")
```

*Join two DataFrames based on a key column.*

## 2. Concatenation

```
df3 = pd.concat([df1, df2], axis=0)
```

*Stack DataFrames vertically (axis=0) or horizontally (axis=1).*

# Reading and Writing Files

## 1. CSV Files

```
df = pd.read_csv("data.csv")
df.to_csv("output.csv", index=False)
```

## 2. Excel Files

```
df = pd.read_excel("data.xlsx")
df.to_excel("output.xlsx", index=False)
```

*Common formats for data exchange.*

# Data Exploration and Utilities

## 1. Sampling

```
df.sample(2)
```

*Randomly select rows.*

## 2. Value Counts and Unique Values

```
df["Name"].value_counts()
df["Name"].unique()
```

## 3. String Operations

```
df["Name"].str.upper()
df["Name"].str.contains("A")
```

## 4. Quick Plotting

```
df["Age"].plot(kind="bar")
```

# Basic Structure of a Class

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        print(f"My name is {self.name}, I am {self.age}
        years old.")
```

Key components:

- `class`: defines a blueprint
- `__init__`: constructor (initialization)
- `self`: represents the current object
- Methods: functions inside a class

# Many ML algorithms are classes

```
from sklearn.linear_model import LogisticRegression
import inspect
print(inspect.getsource(LogisticRegression))
```

Check the details of logistic regression and you will find this is a class.

# Many ML algorithms are classes

```
In [6]: import inspect
   ...:
   ...: print(inspect.getsource(LogisticRegression))
class LogisticRegression(LinearClassifierMixin, SparseCoefMixin, BaseEstimator):
    """
    Logistic Regression (aka logit, MaxEnt) classifier.

    This class implements regularized logistic regression using a set of available
    solvers. **Note that regularization is applied by default**. It can handle both
    dense and sparse input `X`. Use C-ordered arrays or CSR matrices containing 64-bit
    floats for optimal performance; any other input format will be converted (and
    copied).

    The solvers 'lbfgs', 'newton-cg', 'newton-cholesky' and 'sag' support only L2
    regularization with primal formulation, or no regularization. The 'liblinear'
    solver supports both L1 and L2 regularization (but not both, i.e. elastic-net),
    with a dual formulation only for the L2 penalty. The Elastic-Net (combination of L1
    and L2) regularization is only supported by the 'saga' solver.
```

# Summary of Pandas

- DataFrame: main data structure
- Import/export: CSV, Excel
- Selection, filtering, sorting
- Aggregation: groupby, pivot table
- Missing data, applying functions
- Plotting basic charts

# Homework: Iris Dataset Mini Project

**Dataset: Iris**

```
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

iris = load_iris()
df = pd.DataFrame(iris.data,
                  columns=iris.feature_names)
df["species"] = iris.target
```

**Goal:** Practice Python basics, NumPy and Pandas through a small data analysis project.

# Part 1: Basic Exploration

1. Display the first 10 rows.
2. Use:
   - df.info()
   - df.describe()
3. How many samples belong to each species?
4. Are there any missing values?

# Part 2: Feature Engineering & Filtering

**Create New Columns:**

- petal_ratio = petal length / petal width
- sepal_area = sepal length $\times$ sepal width

**Filtering:**

- All samples with sepal length $> 6$
- All samples from species 0 with petal width $< 0.5$

**Sorting:**

- Sort dataset by petal length (descending)

# Part 3: Grouping and Aggregation

1. Compute the mean of each feature grouped by species.
2. Compute both mean and standard deviation.
3. Which species has:
   - Largest average petal length?
   - Smallest average sepal width?
4. Use pivot_table to summarize mean petal length by species.

# Part 4: NumPy Practice

1. Convert feature matrix to a NumPy array.
2. Compute:
   - Column-wise mean
   - Column-wise standard deviation
3. Verify results match pandas output.
4. Reshape the array into shape (150, 4).

# Bonus (Optional)

## 1. Train/Test Split (Simple Version)

```
df_train = df.sample(frac=0.8, random_state=1)
df_test = df.drop(df_train.index)
```

- Compute mean feature values in training set.
- Compare with full dataset mean.

## 2. Write a Function

```
def summarize_by_species(df):
    """
    Return mean and std of each feature
    grouped by species.
    """
```

# Matrix Creation

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
```

- Define two 2x2 matrices A and B

# Matrix Operations

```
C = A + B
D = A @ B    # Matrix multiplication
E = np.linalg.inv(A)
```

- A + B: element-wise addition
- A @ B: matrix multiplication
- np.linalg.inv(A): inverse of A

# Determinant and Trace

```
det_A = np.linalg.det(A)
trace_A = np.trace(A)
print(det_A, trace_A)
```

- det(A) = determinant
- trace(A) = sum of diagonal elements

# Tensor Operations

```
T = np.random.rand(2,3,4)
sum_axis0 = np.sum(T, axis=0)
mean_axis1 = np.mean(T, axis=1)
```

- Create a 3D tensor T
- Compute sum and mean along different axes

# Vectorization vs Python Loops

- Vectorized operations in Numpy are implemented in C and are much faster than Python loops. Avoid explicit for-loops when possible.

```
import numpy as np
import time
N = 10_000_00
x = np.random.rand(N)
# Python loop
start = time.time()
y_loop = np.zeros_like(x)
for i in range(N):
    y_loop[i] = x[i] * x[i]
print("Loop time:", time.time() - start)
# Vectorized
start = time.time()
y_vec = x * x
print("Vectorized time:", time.time() - start)
```

# Vectorized Distance Computation

- Example: compute squared Euclidean distance to a reference vector.
- Compare loop vs vectorized version (100 times faster)

```
X = np.random.rand(10000, 4)
ref = np.random.rand(4)

start = time.perf_counter()
dist_loop = np.zeros(X.shape[0])
for i in range(X.shape[0]):
    dist_loop[i] = np.sum((X[i] - ref)**2)
end = time.perf_counter()
print("Loop time (seconds):", end - start)

start = time.perf_counter()
dist_vec = np.sum((X - ref)**2, axis=1)
end = time.perf_counter()
print("Vectorized time (seconds):", end - start)
```

# Dense vs Sparse: Memory Usage

```
import numpy as np
from scipy.sparse import csr_matrix

dense = np.zeros((10000, 10000))
dense[0, 0] = 1
dense[123, 456] = 2

sparse = csr_matrix(dense)

print("Dense memory (bytes):", dense.nbytes)
print("Sparse non-zeros:", sparse.nnz)
```

- Dense matrix stores all values (including zeros).
- Sparse matrix stores only non-zero entries.

# Constructing Sparse Matrices

```
import numpy as np
from scipy.sparse import csr_matrix

rows = np.array([0, 1, 2])
cols = np.array([1, 3, 4])
vals = np.array([10, 20, 30])

A_sparse = csr_matrix((vals, (rows, cols)), shape=(5, 6))
print(A_sparse)
print(A_sparse.toarray())
```

- CSR (Compressed Sparse Row) format
- Efficient for matrix-vector and matrix-matrix multiplication
- In A_sparse, which values are non-zero?

# Sparse Matrix-Vector Multiplication

```
import numpy as np
from scipy.sparse import random as sparse_random

A_sparse = sparse_random(10000, 10000, density=0.0001, format="csr")
x = np.random.rand(10000, 1)

y = A_sparse @ x
print(y.shape)
```

- Sparse matrix multiplication skips zero entries.
- Much faster and more memory-efficient for very sparse data.

# Dense vs Sparse: Speed Comparison

```
from scipy.sparse import random as sparse_random
n, density = 5000, 0.0002

A_dense = np.random.rand(n, n)
A_dense[A_dense > density] = 0  # make it sparse-like
x = np.random.rand(n, 1)
A_sparse = sparse_random(n, n, density=density, format="csr")

start = time.perf_counter()
y_dense = A_dense @ x
print("Dense time:", time.perf_counter() - start)

start = time.perf_counter()
y_sparse = A_sparse @ x
print("Sparse time:", time.perf_counter() - start)
```

- For very sparse data, sparse multiplication is much faster. Dense computation wastes time on zeros (20 times faster)

# When to Use Sparse Matrices?

- Matrix is mostly zeros (e.g.,$>$ 95% zeros).
- Matrix is very large (memory becomes a bottleneck).
- Operations are mainly matrix-vector or matrix-matrix multiplication.
- Common in:
  - Natural language processing
  - Graph algorithms
  - Recommender systems

# Load Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
df["species"] = df["target"].map({0:"setosa",1:"versicolor",2:"
    virginica"})
```

- Create DataFrame and add species column

# View Data

```
df.head()
df.info()
df.describe()
```

- head(): first five rows
- info(): data structure information
- describe(): descriptive statistics

# Filtering Rows

```
setosa = df[df["species"]=="setosa"]
versicolor = df[df["species"]=="versicolor"]
```

- Filter data by species

# Column Selection

```
sepal_data = df[["sepal length (cm)", "sepal width (cm)"]]
print(sepal_data.head())
```

- Select specific columns for analysis

# Add New Columns

```
df["sepal_area"] = df["sepal length (cm)"] * df["sepal width (cm)"]
df["petal_area"] = df["petal length (cm)"] * df["petal width (cm)"]
```

- Compute sepal and petal area

# Sorting Data

```
df_sorted = df.sort_values(by="sepal_area", ascending=False)
print(df_sorted.head())
```

- Sort data by sepal_area

# Groupby and Aggregation

```
grouped = df.groupby("species")[["sepal_area","petal_area"]].mean()
print(grouped)
```

- Group by species and compute mean area

# Apply Function

```
df["sepal_ratio"] = df.apply(
    lambda row: row["sepal length (cm)"] / row["sepal width (cm)"],
        axis=1)
```

- Compute sepal length/width ratio

# Boolean Indexing

```
long_sepals = df[df["sepal length (cm)"] > 5.0]
print(long_sepals.head())
```

- Filter rows with sepal length greater than 5.0

# Value Counts

```
species_counts = df["species"].value_counts()
print(species_counts)
```

- Count number of samples per species

# Correlation Matrix

```
cor_matrix = df.iloc[:,0:4].corr()
print(cor_matrix)
```

- Compute correlation between features

# Standardization

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df.iloc[:,0:4])
print(df_scaled[:5])
```

- Standardize features for analysis

# Matrix Multiplication Example

```
X = df_scaled
cov_matrix = X.T @ X / X.shape[0]
print(cov_matrix)
```

- Compute covariance matrix from standardized data

# Eigenvalues and Eigenvectors

```
eig_vals, eig_vecs = np.linalg.eig(cov_matrix)
print("Eigenvalues:", eig_vals)
```

- Eigen decomposition for PCA

# Tensor Statistics

```
T = np.random.rand(150,4,3)  # Simulate 3 measurements
mean_tensor = np.mean(T, axis=0)
print(mean_tensor)
```

- Compute statistics on multi-dimensional array

# Save Processed Data

```
df.to_csv("iris_processed.csv", index=False)
```

- Save processed DataFrame to CSV

# Load Saved Data

```
df_new = pd.read_csv("iris_processed.csv")
print(df_new.head())
```

- Load CSV to verify

# Summary Statistics per Class

```
summary = df.groupby("species").agg({
    "sepal length (cm)": ["mean","std"],
    "sepal width (cm)": ["mean","std"],
    "petal length (cm)": ["mean","std"],
    "petal width (cm)": ["mean","std"]
})
print(summary)
```

# Top N Samples by Area

```
top3 = df.nlargest(3,"sepal_area")
print(top3)
```

# Filter Using Multiple Conditions

```
subset = df[(df["sepal length (cm)"]>5.0) & (df["petal length (cm)"
    ]<3.0)]
print(subset)
```

# Random Sampling

```
sample_df = df.sample(n=10, random_state=42)
print(sample_df)
```

# Reset Index

```
df_reset = df.reset_index(drop=True)
print(df_reset.head())
```

# Pivot Table Example

```
pivot = df.pivot_table(index="species", values="sepal_area", aggfunc
    ="mean")
print(pivot)
```

# Chaining Operations

```
result = (df[df["sepal_area"]>15]
          .groupby("species")["petal_area"]
          .mean())
print(result)
```

# Final Remarks

- Demonstrated matrix and tensor operations with Numpy
- Used Pandas to load, manipulate, and analyze Iris dataset
- Prepared foundation for data visualization and advanced analysis